

C++ & QT Day3-hw3 AI 보고서

로봇 21기 예비단원 최윤서

1. 마우스 클릭 이벤트

- 공식 문서에 적혀있는 `MouseEvent` 를 찾아보았는데 어떤 헤더파일을 사용하고 `CMakeList`를 어떻게 수정해야하는지 이런 내용만 적혀있어서 구체적인 구현 방법을 찾지 못했습니다.
- 따라서, `MouseEvent` 말고도 다른 기능이 있을까 싶어 조연을 구했습니다.

Upgrade

QT creator에서 C++로 마우스 클릭 입력받으려고하는데 어떤 로직 사용해야
돼? 코드 짜주는말고 어떻게 하는지 방법만 알려줘

네 코드 기준으로는 `mousePressEvent()` 를 직접 `MainWindow` 에 구현하는 방식보다, `MapCell` 에서 클릭을 감지하고 `Signal & Slot`으로 `MainWindow` 에 전달하는 방식이 맞습니다.

전체 로직

```
사용자가 격자(MapCell)를 클릭
↓
MapCell에서 마우스 클릭 감지
↓
clicked(row, col) Signal 발생
↓
MainWindow가 Signal을 받음
↓
onCellClicked(row, col) 실행
↓
currentClickMode 확인
↓
| 장애물 모드 | 시작점 모드 | 목표점 모드 |
| 장애물 설정 | 시작점 설정 | 목표점 설정 |
```

Upgrade

QT creator에서 이미 MapCell 을 생성할 때 각 셀의 clicked Signal을 MainWindow::onCellClicked() 에 연결하고 있습니다. Pasted text

네 코드에서 각각의 역할

1. `MapCell`

실제 마우스 클릭을 감지하는 객체입니다.

즉,

"사용자가 몇 번째 행, 몇 번째 열의 셀을 클릭했다."

를 알아내는 역할입니다.

2. `clicked(row, col)`

클릭했다는 사실과 **클릭한 셀의 위치**를 전달하는 Signal입니다.

예를 들어 사용자가 3행 5열을 클릭했다면 개념적으로:

```
clicked(3, 5)
```

가 발생하는 구조입니다.

3. `connect()`

MapCell 에서 발생한 클릭 Signal을 MainWindow 의 처리 함수와 연결합니다.

네 코드에서는 이미 이 역할을 하고 있습니다. Pasted text

4. `onCellClicked(row, col)`

실제 클릭 후의 동작을 결정하는 곳입니다.

네 코드에서는 `currentClickMode` 를 확인해서 동작을 분기합니다. Pasted text

예를 들어:

```
currentClickMode == DrawObstacle
    → 장애물로 변경

currentClickMode == SetStart
    → 시작점으로 변경

currentClickMode == SetGoal
    → 목표점으로 변경

currentClickMode == None
    → 아무 동작도 하지 않음
```

따라서 네가 새로 마우스 클릭 기능을 만들 때

네 프로젝트 구조에서는 다음 순서로 생각하면 됩니다.

MapCell 에서 클릭 감지 → Signal로 위치 전달 → `connect()` → `onCellClicked()` 에서 처리

그리고 단순히 QPushButton을 클릭하는 경우에는 `clicked()` Signal을 사용하면 되고, 지금처럼 격자 하나하나의 클릭 위치를 받아야 하는 경우에는 네 코드의 MapCell + Signal/Slot 구조가 적합합니다.

- 느낀점)

- 결국 격자점이기 때문에 굳이 마우스 클릭을 직접 받지않아도 그 셀이 클릭되면 애초에 마우스 클릭을 받아들 수 있기 때문에 굉장히 참신한 아이디어라고 생각했습니다.
- 지금까지 과제를 진행하며 계속 코드를 대신 짜주지말고 방법만 알려달라고 한 후 구글링 예제를 찾아서 해결하고있는데 시간이 오래 걸리긴하지만 확실하게 알고리즘을 이해하고 넘어가는 것이 중요하다는 것을 느꼈습니다.

2. 격자점 색 바꾸기

- 마우스 클릭 문제를 해결했으니 장애물, 시작점, 도착점 설정 문제는 해결했지만 GUI에서 클릭 시 색을 어떻게 바꿔야할지 모르겠어서 조언을 구했습니다.

마우스 클릭했을 때 그 셀의 색깔 바꾸는 법 알려줘

Qt에서 격자 셀의 색을 바꾸는 방법은 **셀 위젯의 배경색을 변경**하는 거야.

네가 MapCell 을 QWidget 기반으로 만들었다면 가장 간단한 방법은 **Style Sheet(QSS)**를 사용하는 거야.

1. 셀의 색을 변경

예를 들어 클릭된 셀을 검은색으로 만들고 싶다면:

```
cell->setStyleSheet("background-color: black;");
```

빈 셀로 되돌리고 싶다면:

```
cell->setStyleSheet("background-color: white;");
```

즉,

```
클릭
↓
현재 상태 확인
↓
Empty → Obstacle
↓
검은색으로 변경
↓
다시 클릭
↓
```

```
Obstacle → Empty
↓
흰색으로 변경
```

2. 네 구조에서는 어디서 바꾸면 되나?

현재 네 코드처럼 MainWindow에서 클릭한 셀의 row, col을 받는 구조라면,

onCellClicked(row, col) 안에서 해당 셀의 색을 변경하면 된다.

다만 중요한 점이 하나 있어.

네 코드에는 MapCell 객체를 cellStorage에 저장하고 있으므로,

```
cellStorage[row][col]
```

을 이용해서 클릭된 실제 셀 위젯을 찾은 다음 색을 변경하면 된다.

개념적으로는:

```
onCellClicked(row, col)
↓
cellStorage[row][col] 찾기
↓
현재 상태 확인
↓
Obstacle이면 검은색
Empty이면 흰색
↓
```

3. 추천하는 방식

장애물 여부를 판단하는 데이터와 화면 색상을 같이 관리하는 게 좋다.

예를 들어:

```
mapData[row][col] = Empty
→ 셀 흰색

mapData[row][col] = Obstacle
→ 셀 검은색
```

그러면 나중에 A*나 Dijkstra에서 장애물을 판단할 때도 mapData를 그대로 사용할 수 있어.

즉, 색깔 자체를 장애물 여부로 사용하는 게 아니라 mapData가 실제 상태이고, 색깔은 그 상태를 보여주는 용도로 생각하면 된다.

- 느낀점)

- 실제로 구현하면서 클릭한 격자의 행과 열을 구분하고 해당 격자의 상태를 변경하는 과정이 필요하다는 것을 알게됐습니다.
- Qt의 Signal과 Slot을 이용하면 각각의 격자에서 발생한 클릭 이벤트를 MainWindow로 전달하여 처리할 수 있다는 점을 배웠습니다.